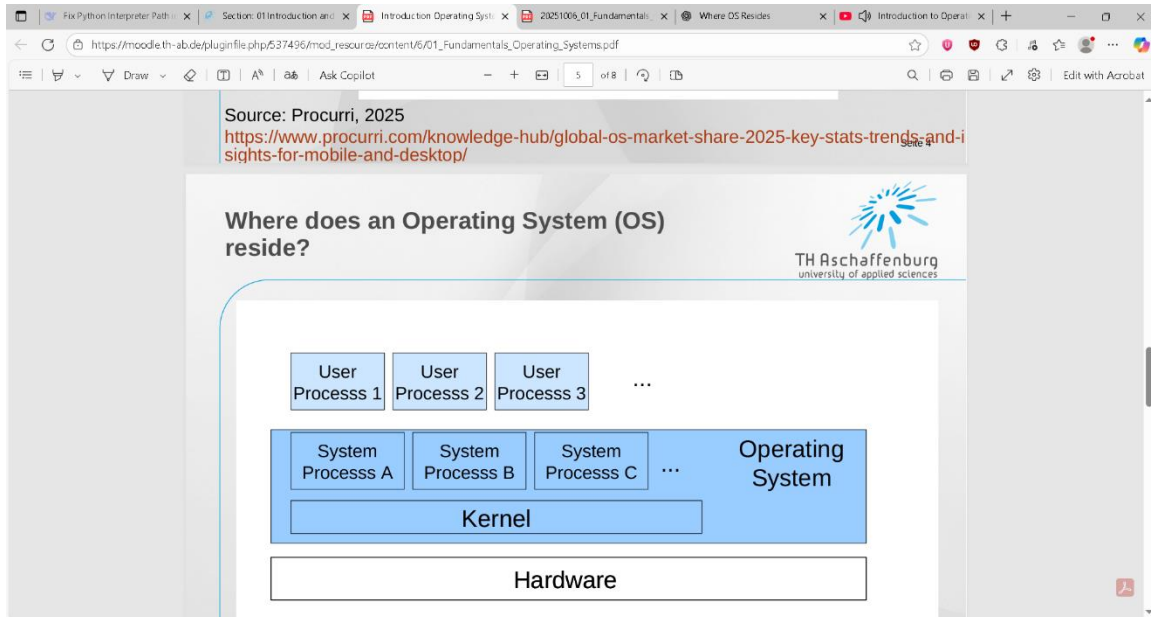


## Where does an OS reside:



...

**1) Hardware (the lowest bottom): CPU, RAM, GPU, Disks like SSD HDD, IO Devices, etc...**

**N.B** the OS files are stored on HDD and SDD whenever the system is off.

The CPU executes instruction that comes only directly from the RAM it cannot execute instructions from the SSD HDD or on its own.

### **2) Boot phase — how the OS gets into RAM:**

- The bootloader loads the kernel image from disk into RAM and transfers control to it.
- The BIOS starts once you hit the start button it checks that the hardware is all set and runs the hardware, then runs the bootloader,
- bootloader finds the kernel image on the hard drive then pass a kernel image to the RAM of the kernel, afterwards the CPU reads the kernel and the kernel then takes control and runs the operating system files and services.

### 3) Kernel: the heart of the OS:

- It schedules processing, what program can get to the CPU and for how long, like multi-threading priorities.
- Kernel manages the memory for the system RAM and makes sure each program gets its own safe storage.
- The kernel communicates with hardware devices through drivers (translators) and handle interrupts (call for actions) when a button is clicked.
- The kernel manages file systems
- Kernel has a high level of privilege and operates at a high level in the RAM.

### 4) System processes / OS services (inside the OS box but above kernel):

- These run in user space in many systems but are considered part of the OS's functionality (e.g., systemd, daemons, file system servers, network services). Some OS services (like device drivers) may be part of the kernel (monolithic kernel) or run as separate processes (microkernel).
- They provide higher-level services built on top of kernel primitives (e.g., mounting file systems, network stacks, authentication services).
- They may be started by the kernel at boot and kept running to service user programs.

### 5) User Application:

#### What They Are

- They live in user space, which is a special part of memory where programs can run safely without touching critical system parts.
- These applications have less privilege and more restrictions means that they cannot do tasks like writing directly on the Hard drive, for example if you hit the save button on Microsoft word it goes to the kernel and asks for permission then the kernel itself is the one the is writing on the hard drive.
- **Virtual Memory / Isolation:** Each user process gets its own virtual address space. On the RAM and if it runs out of this allocated space it crashed without crashing the OS

## **Monolithic VS Micro Operating Systems:**

### **Monolithic OS:**

An architecture for operating systems that lets the OS services have the same privilege level and share the same memory in RAM as its of the kernel making the system faster and giving these services the access to deal with the same hardware that the kernel can. Through less IPC messages this makes the OS faster. OS services run in kernel mode and can access hardware directly.

### **MicroKernel OS:**

The OS services operate at the user level making it separate from the kernel space and don't have access to the same hardware and storage. Making the OS slower but safer, also for the OS services to change something on the hardware it should send a IPC message to the kernel.

## What is an OS for? What are the essential features of an OS?

### What is an OS for:

An (OS) is software that manages your computer's hardware and software resources and provides a platform for applications to run.

Think of the OS as a middleman between your programs and the hardware. Without an OS, programs would have to talk directly to the CPU, memory, disk, and other devices — which would be extremely complicated and unsafe.

Key purposes of an OS:

- **Resource Management**
  - Manages CPU, memory, storage, and I/O devices.
  - Decides which program gets what resources and when.
- **Hardware Abstraction**
  - Provides a simplified interface for programs to use hardware without needing to know all the low-level details.
  - Example: You just call `read()` in your program, and the OS handles disk access.
- **Program Execution**
  - Loads, runs, and terminates programs.
  - Handles multitasking so multiple programs can run “simultaneously.” (Multi-threading)
- **Security & Protection**
  - Prevents programs from accessing memory or hardware they shouldn't.
  - Protects the system from crashes or malicious software.
- **User Interface**
  - Provides a way for users to interact with the computer, e.g., GUI (Windows, macOS) or command-line (Linux terminal).

## What are the essential features of an OS:

### **Process Management**

- Creates, schedules, and terminates processes (programs in execution).
- Handles multitasking and assigns CPU time using a scheduler.

### **Memory Management**

- Allocates memory for applications to processes safely.
- Provides virtual memory so each process thinks it has its own full memory.

### **File System Management**

- Organizes and controls how data is stored and retrieved from disks.
- Provides directories, permissions, and metadata.

### **Device Management / Drivers**

- Communicates with hardware devices like printers, keyboards, disks, and network cards.
- Handles interrupts from devices and provides an abstraction layer for programs.

### **Security & Access Control**

- Ensures that users and programs can only access resources they are allowed to.
- Protects kernel memory and prevents unauthorized hardware access.

### **Networking & Communication**

- Provides services for sending and receiving data over networks.
- Supports protocols like TCP/IP and manages sockets.

### **User Interface**

- Offers ways for users to interact with the system.
- Could be a GUI (Windows/macOS) or CLI (Linux/Unix).

### **Error Detection & Recovery**

- Monitors for hardware/software errors and takes corrective action.
- Example: closing a crashed program without crashing the whole system.

